RL PCB

An AI design assistant for KiCad that finds parts, calculates values, places components and verifies wiring, and deliberately never draws the wires itself.

Dalton Omondi · Kenya · DTMES 2026 Africa Electronics Innovation Challenge — Project Competition (Built Stage) · September 2026

1. Problem

Designing a printed circuit board needs expertise that is scarce and slow to acquire. A beginner has to choose parts, derive component values from datasheets, connect hundreds of pins correctly and lay out a board that meets electrical rules. One wrong connection means a dead board, and every new fabrication run costs weeks and money that students and small hardware teams in Africa rarely have. Most have no senior engineer to review their work.

General AI chatbots don't fill this gap. They draw schematics badly, producing "ghost" wires that look connected but aren't, and they invent part numbers that don't exist. The help sounds confident but can't be trusted.

2. Solution and design

> **Core idea.** The AI does everything around the drawing (research, parts, calculations, placement, instructions and verification), and the human draws the wires. The AI's *design plan* is a contract. The human's drawing is the source of truth. RL PCB checks one against the other, so every claim it makes can be verified.

1 · Plan	2 · Review	3 · Place	4 · Human wires	5 · Verify → Board
Requirements → architecture → parts from the user's own KiCad libraries → values from shown formulas.	Electrical rule check of the *intent* before drawing: power pins, floating inputs, decoupling, conflicts.	Symbols grouped around the pins they serve, on hierarchical sheets; exact per-pin wiring instructions.	The user draws the wires in KiCad. by design	Drawing vs plan (missing, shorted, near-miss wires); stackup, trace rules, placement, autorouting, DRC.

Architecture

Chat panel inside KiCad · **Claude Desktop / any MCP client** → one registry of ~60 specialised tools

KiCad engine Reads .kicad_sch/.kicad_pcb directly (KiCad 4–10). Re-implements KiCad's connectivity rules: wires, junctions, labels, power symbols, hierarchical sheets, buses.	**Design** Plan & review, symbol placement (no wires), wiring guide, verification, ghost-wire detector.	**Board** Stackup advisor, net classes, two-sided placement of passives, file & live moves, Freerouting autorouting, DRC, fabrication outputs.
Library index Full-text search over every symbol and footprint installed (22,860 symbols and 15,450 footprints on the test laptop).	**Knowledge** 34 engineering guides and reference architectures, plus the user's own PDFs and datasheets.	**Reference corpus** 136 open-source designs (3,518 parts): "how do real designs wire this chip?"

The model reasons and explains; the tools do everything that must be exact (coordinates, netlists, geometry, calculations). Live control uses KiCad 9/10's official API; everything else works on the files, so KiCad 7/8 is covered too.

3. Innovation

- **A division of labour instead of generation.** AI tools for electronics usually try to generate whole circuits and fail silently. RL PCB never draws wires and checks every result against an explicit plan.
- **Verification backed by KiCad.** Its connectivity engine gives results identical to KiCad's own netlister (see §4), which is why its checks can be relied on.
- **Checks designed for weaker models.** Every step reports a machine-checkable signal (errors, % connections verified, overlaps, unrouted count). The next version adds pre-verified circuit blocks so that smaller, cheaper models can use it offline. It all runs locally on a laptop with free tools (KiCad, Freerouting).

4. Results

4.1 Connectivity engine vs KiCad's own netlister

Test set	What was compared	Result
13 official KiCad 7 demo projects (incl. hierarchical, bus-heavy designs)	Every net vs `kicad-cli` netlist export	**13 / 13 identical**
VESC motor controller (open source, KiCad 4 legacy format)	Every net vs the project's own netlist	**identical**
KiCad 9/10 demos (Jetson carrier, CM5, VME)	Schematic nets vs the boards' nets	**match***
Footprint flip to bottom side (file mode)	Pads, graphics, text vs KiCad's own flip, 12 footprint types × 4 rotations (48 footprints)	**exact**

* apart from items that are stale in the demo files themselves.

4.2 End-to-end run on a new design: drone flight controller

Brief: 36×36 mm flight controller with 30.5 mm M3 mounting and a 2–6S LiPo input. Parts: STM32F405 MCU, ICM-20602 IMU (SPI), BMP280 barometer (I^2C), 128 Mbit log flash, TPS5430 5 V/3 A buck regulator, 3.3 V LDO, USB-C with ESD protection, ESC / receiver / video transmitter / SWD connectors.

Stage	Result
Parts chosen	**61 parts, all from stock KiCad libraries**
Values calculated (formula shown)	Buck feedback 9.31 kΩ / 3.01 kΩ → 4.998 V · crystal loads 2·(12−3) = 18 pF · battery divider 10 k/1 k (25.2 V → 2.29 V) · inductor ≈13.4 µH → 15 µH
Plan review before drawing	**0 errors · 0 warnings**
Schematic	4 hierarchical sheets, symbols grouped around their pins, **no wires drawn**
Verification of the finished drawing	**174 / 174 connections (100%)**
Cross-check with KiCad's netlister	**79 / 79 nets agree**
Deliberate mistakes injected	**2 / 2 caught with location**
Board: 4 layers, 36×36 mm, two-sided	**65 footprints · 0 courtyard overlaps**
Autorouting (Freerouting, 3 min)	**88–91% routed · 0 clearance errors from routing**

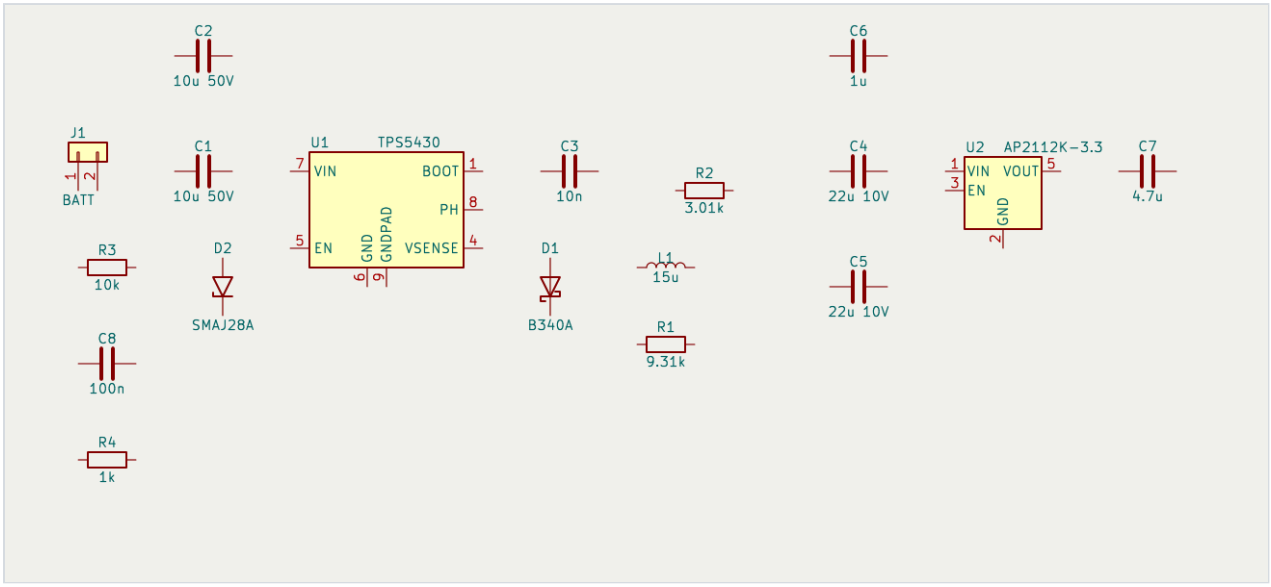
4.3 Mistake detection (actual output)

```
ERROR  short: Planned nets I2C1_SCL, I2C1_SDA are connected together in the drawing.
       I2C1_SCL: U3.61; I2C1_SDA: U5.3, R9.2            (SDA and SCL swapped on the barometer)
WARN   global label 'SPI1_MISO' is not attached to a wire or pin; nearest pin is
       U4.4 (SAO/SD0) 0.64 mm away (near-miss: move the label onto that pin end)
```

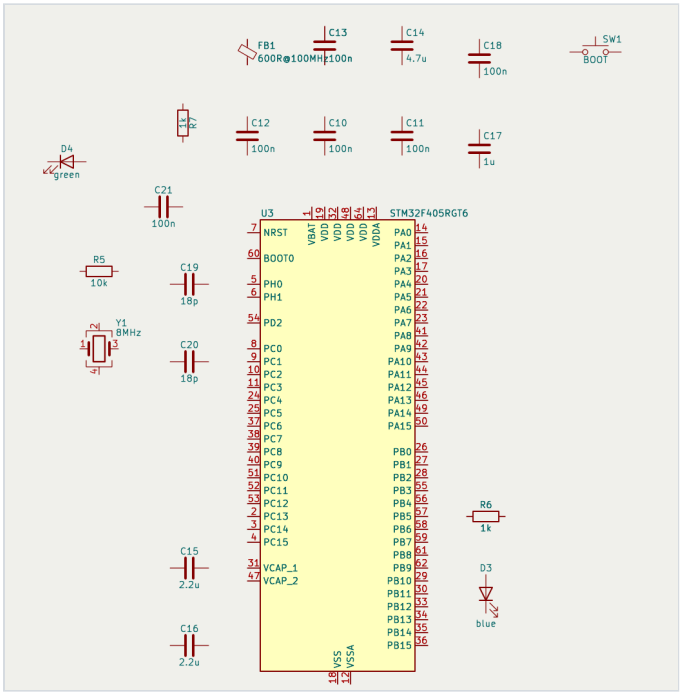
4.4 Wiring instructions (actual output, power sheet)

```
- +3V3, +5V, GND  [power symbols]      - VBAT [global labels: spans sheets]
- BUCK_SW  Draw a wire from U1.8 (PH) at (78.74, 45.72) to C3.2 at (93.98, 40.64)
           Draw a wire from C3.2 at (93.98, 40.64) to L1.1 at (99.06, 53.34)
           Draw a wire from L1.1 at (99.06, 53.34) to D1.1 (K) at (87.63, 59.69)
```

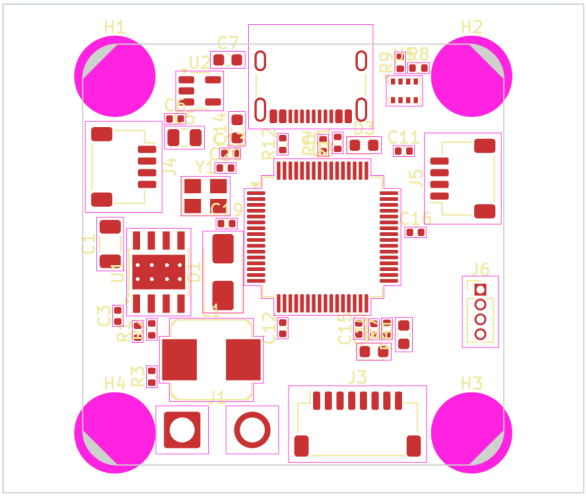
5. The design, stage by stage



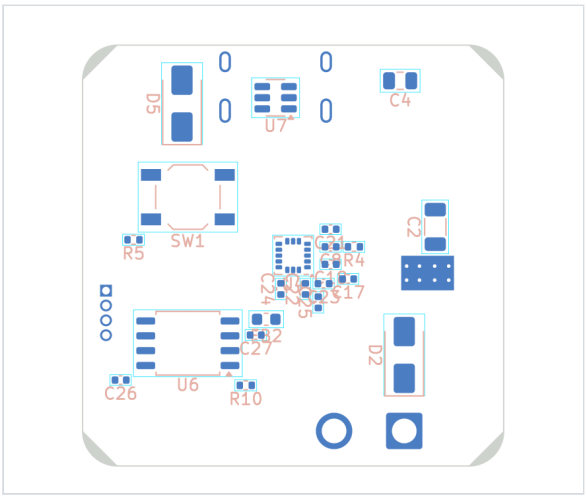
Power sheet as placed by RL PCB: battery input, TVS, TPS5430 buck with calculated feedback divider, 3.3 V LDO, battery-sense divider. Symbols and values only; the wires are for the human to draw.



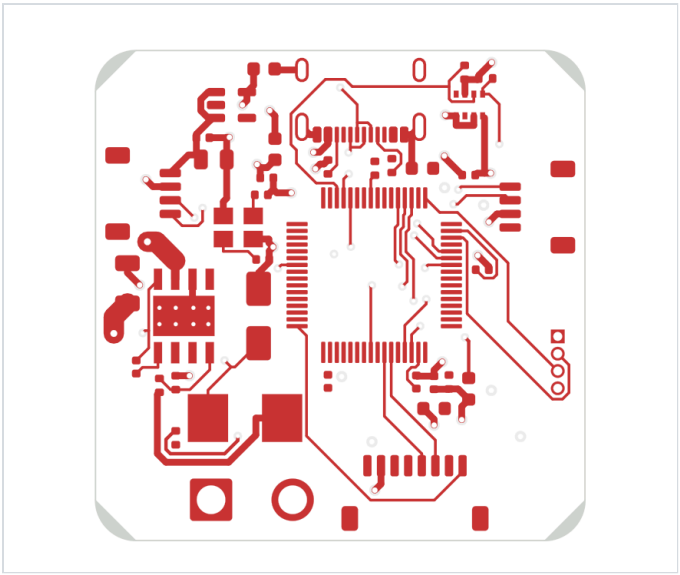
MCU sheet: STM32F405 with decoupling, VCAP, VDDA filter, 8 MHz crystal and load capacitors, reset, boot and LEDs, grouped by the pins they serve.



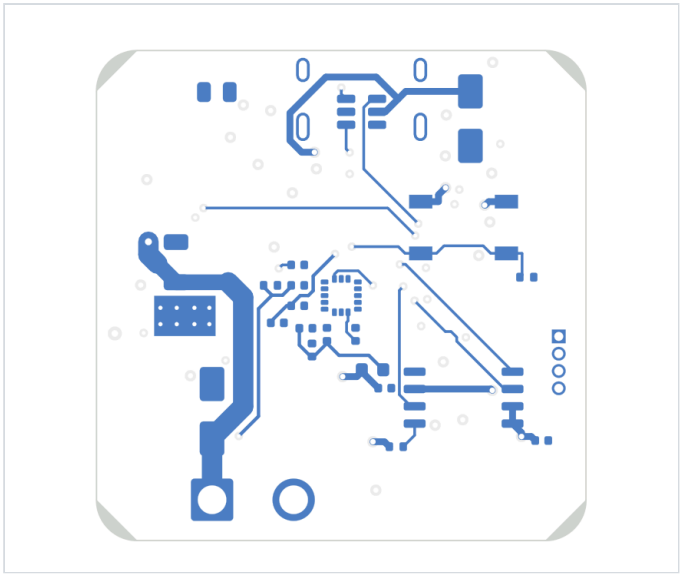
Board placement, top: connectors on the edges, MCU centred, buck regulator in the corner next to the battery pads.



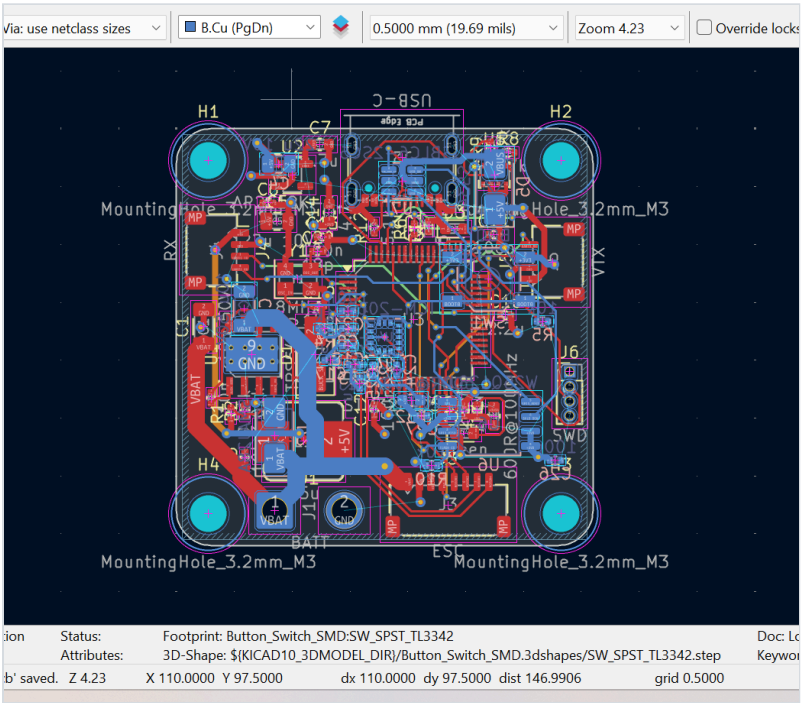
Bottom (mirrored view): IMU at the board centre, flash, USB ESD, boot button, and decoupling under the pins it serves.



After autorouting, top copper. Power paths and the switch-node loop are marked for manual review.



After autorouting, bottom copper.



The routed board open in KiCad 10.0.6 on my own laptop (Windows). RL PCB is installed there: library index built, KiCad API connected, tools available in Claude Desktop.

6. Impact and applications

- **Students, makers and early hardware startups** designing controllers, sensors, power supplies, motor drivers, drones and industrial I/O. RL PCB acts as the reviewer they don't have, cutting failed board revisions.
- **Teaching.** Every decision is explained with its formula or source, so it doubles as a tutor for university and technical-college electronics courses.
- **Local design capacity.** It lowers the barrier to designing electronics in Africa, with fabrication abroad where needed, instead of importing finished designs.

7. Honest status and next steps

- The drone board is a **design generated and verified by the tool**. It has not been fabricated or reviewed by an experienced engineer. Open items flagged for datasheet review: IMU reserved pins, TVS orientation, inductor saturation current ≥ 3 A, and USB back-feed into the battery rail.
- About 10% of connections are left for manual routing, as are the critical paths (regulator hot loop, USB, IMU SPI). Remaining DRC items come from fine-pitch footprints under the default 0.2 mm clearance.
- Built with extensive AI assistance (Claude) for implementation. I defined the problem, the no-wires architecture and the requirements, and tested it on my own KiCad setup.

Next step	Purpose
Library of pre-verified circuit blocks (MCU core, regulators, USB, sensors)	Lets smaller, cheaper and offline models design reliably: they choose blocks instead of wiring every pin
More automatic checks (part voltage/current ratings, board fit, back-feed paths, missing pull-ups)	Turns expert "noticing" into pass/fail signals
Multi-board systems (connector pinouts across harnesses, power budget)	Products like printers and industrial machines are several boards
Public error-detection benchmark and model comparison	A measured detection rate across hundreds of injected mistakes
Fabricate and bring up the drone board	Close the loop from design to working hardware